UNIT-IV
LOGIC BASED AND ALGEBRIC MODELS

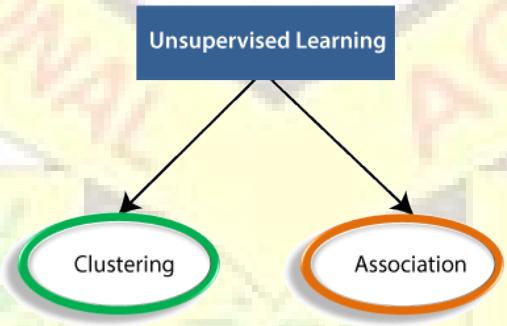
Unsupervised Learning

- Unsupervised learning is a type of machine learning in which models are trained using unlabeled dataset and are allowed to act on that data without any supervision.
- In unsupervised learning, the machine is trained on a set of unlabeled data, which means that the input data is not paired with the desired output. The machine then learns to find patterns and relationships in the data.
- Unsupervised learning cannot be directly applied to a regression or classification problem because unlike supervised learning, we have the input data but no corresponding output data.
- The goal of unsupervised learning is to *find the underlying structure of dataset, group that data according to similarities, and represent that dataset in a compressed format*.
- Unsupervised learning is often used for tasks such as clustering, dimensionality reduction, and anomaly detection.

Types of Unsupervised Learning Algorithm:

The unsupervised learning algorithm can be further categorized into two types of problems:

- **Clustering**: Clustering is a method of grouping the objects into clusters such that objects with most similarities remains into a group and has less or no similarities with the objects of another group. Cluster analysis finds the commonalities between the data objects and categorizes them as per the presence and absence of those commonalities.
- **Association**: An association rule is an unsupervised learning method which is used for finding the relationships between variables in the large database. It determines the set of items that occurs together in the dataset. Association rule makes marketing strategy more effective. Such as people who buy X item (suppose a bread) are also tend to purchase Y (Butter/Jam) item. A typical example of Association rule is Market Basket Analysis.



Examples of Unsupervised Learning Algorithms:

- ✓ KNN (k-nearest neighbors)
- ✓ Hierarchal clustering
- ✓ K-means clustering
- ✓ Anomaly detection
- ✓ Principal Component Analysis
- ✓ t-SNE (t-Distributed Stochastic Neighbor Embedding)
- ✓ Independent Component Analysis
- ✓ Singular value decomposition

Application of Unsupervised learning

- ✓ Image analysis

- ✓ Anomaly detection
- ✓ Scientific discovery
- ✓ Recommendation systems
- ✓ Customer segmentation

Distance Based Models:

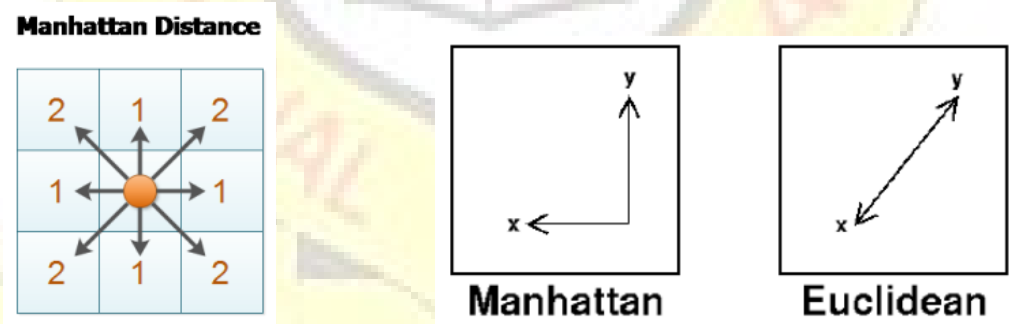
- In Distance-based models, we can use the geometric notion of distance to represent similarity.
- If the distance between two instances is small then the instances are similar in terms of their feature values, and so nearby instances would be expected to receive the same classification or belong to the same cluster.
- In a Cartesian coordinate system, distance can be measured by ***Euclidean distance***, which is the square root of the sum of the squared distances along each coordinate.

$$Euclidean\ distance = Dist_2(x,y) = \sqrt{\sum_{i=1}^{d}(x_i - y_i)^2} = \sqrt{(x-y)^T(x-y)}$$

- A very simple distance-based classifier is ***nearest-neighbour classifier***. (Ex. k-nearest neighbor). KNN identifies the nearest points or groups for a query point. Common distance metrics include Euclidean distance, Manhattan distance, Minkowski Distance, Mahalanobis distance.
- For instance, instead of using Euclidean distance, which can be sensitive to outliers, other distances can be used such as ***Manhattan distance***, which sums the distances along each coordinate.

$$Manhattan\ Distance = Dist_1(x,y) = \sum_{i=1}^{d}|x_i - y_i|$$

- In chess, the distance between squares on the chessboard for rooks is measured in Manhattan distance.



- The Minkowski distance of order p > 0 is defined as:

$$Minkowski\ distance = Dist_p(x,y) = \left(\sum_{i=1}^{d}|x_i - y_i|^p\right)^{1/p} = \|x-y\|_p$$

- To classify a new instance, we retrieve from memory the most similar training instance (i.e., the training instance with smallest Euclidean distance from the instance to be classified), and simply assign that training instance's class.
- The Hamming distance between two strings of equal length is the number of positions at which the corresponding symbols are different i.e., it measures the minimum number of substitutions required to change one string into the other.

  Example:- The Hamming distance between:
  - ✓ "karolin" and "kathrin" is 3.
  - ✓ 1011111 and 1001011 is 2.

- ✓ 9173845 and 9233745 is 3.

- Some examples of Distance based models: k-nearest neighbor, k-means clustering, and hierarchical clustering.

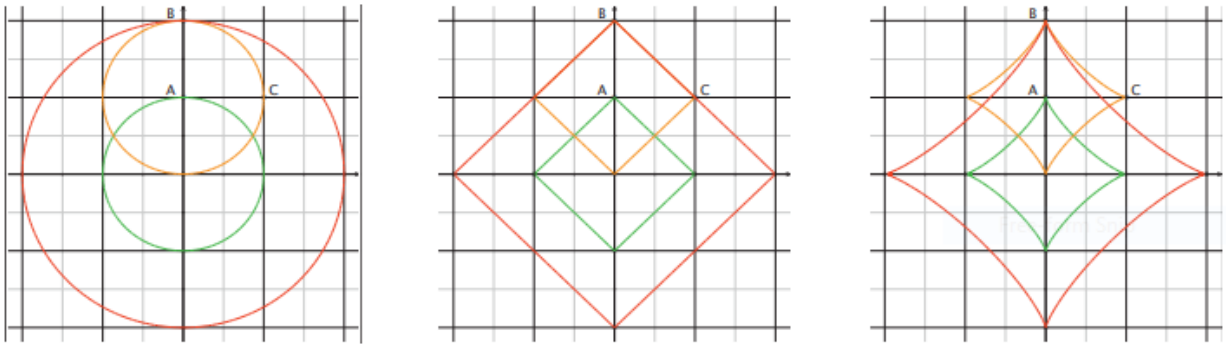


Fig:- (**left**) With Euclidean distance (i.e., Minkowski distance of order p = 2)
(**middle**) With Manhattan distance (p = 1) and
(**Right**) With p < 1 (here, p = 0.8).

Distance metric:

In the context of Machine learning, the concept of distance is not based on merely the physical distance between two points. Instead, we could think of the distance between two points considering the **mode of transport** between two points. Travelling between two cities by plane covers less distance physically than by train because a plane is unrestricted. Similarly, in chess, the concept of distance depends on the piece used – for example, a Bishop can move diagonally.

Given an instance space X, a distance metric is a function Dis: $X \times X \rightarrow \mathbb{R}$ such that for any x, y, z $\in X$.

1. distances between a point and itself are zero: Dis(x, x) = 0
2. all other distances are larger than zero: if x ≠ y then Dis(x, y) > 0
3. distances are symmetric: Dis(y, x) = Dis(x, y)
4. detours cannot shorten the distance: Dis(x, z) ≤ Dis(x, y) + Dis(y, z).

Neighbours and exemplars:

- Distance is applied through the concept of **neighbours** and **exemplars**.
- Neighbours are points in proximity with respect to the distance measure expressed through exemplars.
- Exemplars are either **centroids** that find a centre of mass according to a chosen distance metric or **medoids** that find the most centrally located data point.
- The most commonly used centroid is the arithmetic mean, which minimises squared Euclidean distance to all other points.
- Formulating the model in terms of a number of prototypical instances is known as exemplars and defining the decision rule in terms of the nearest exemplars or neighbours.
- The **centroid** represents the geometric centre of a plane or figure, i.e., the arithmetic mean position of all the points in the figure from the centroid point. This definition extends to any object in *n*-dimensional space: its centroid is the mean position of all the points.
- **Medoids** are similar in concept to means or centroids. Medoids are most commonly used on data when a mean or centroid cannot be defined. They are used in contexts where the centroid is not representative of the dataset, such as in image data.
- Finding a medoid requires us to calculate, for each data point, the total distance to all other data points, in order to choose the point that minimises it.
- Regardless of the distance metric used, this is an $O(n^2)$ operation for n points, so for medoids there is no computational reason to prefer one distance metric over another.

- An alternative, distance-based way to classify instances without direct reference to a decision boundary is by the following decision rule: if x is nearest to $\mu^{\oplus}$ then classify it as positive, otherwise as negative; or equivalently, classify an instance to the class of the nearest exemplar.
- Examples of distance-based models include the **nearest-neighbour** models, which use the training data as exemplars – for example, in classification. The **K-means clustering** algorithm also uses exemplars to create clusters of similar data points.

Nearest Neighbours Classification

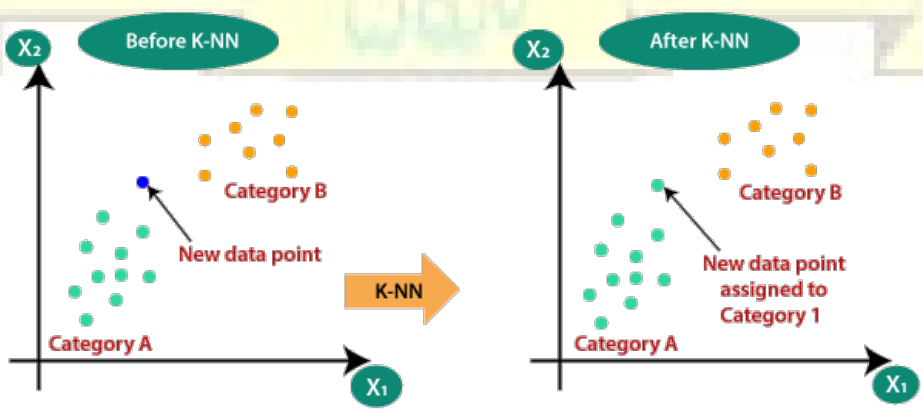
- In fact, the most commonly used distance-based classifier is more straightforward and it simply uses each training instance as an exemplar (i.e., nearest neighbor).
- Consequently, 'training' this classifier requires nothing more than memorizing the training data.

K-Nearest Neighbor(KNN) Algorithm

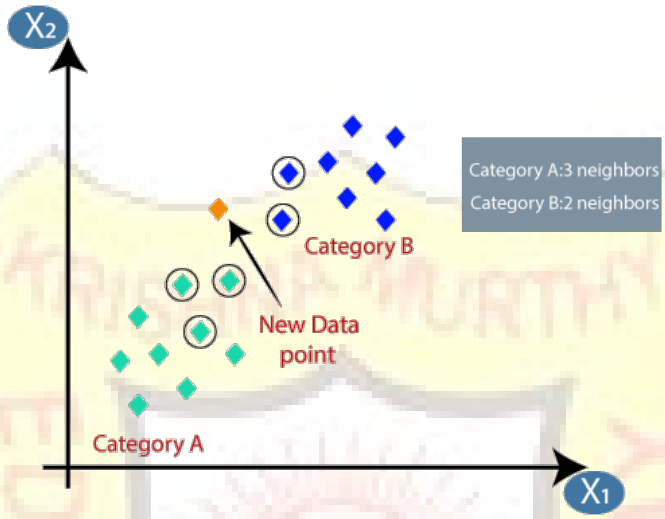
- The K-Nearest Neighbors (KNN) algorithm is a supervised machine learning method employed to tackle classification and regression problems.
- K-NN algorithm is a versatile and widely used machine learning algorithm that is primarily used for its simplicity and ease of implementation.
- The "k" in KNN is a parameter that refers to the number of nearest neighbors to include in the majority of the voting process.
- The k-nearest neighbors of an instance are defined in terms of the standard Euclidean distance.

$$\text{Euclidean distance} = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

- Other distance metrics used in k-NN algorithm are: Manhattan Distance, Minkowski Distance, Hamming Distance and Mahalanobis distance.
- The class or value of the data point is then determined by the majority vote or average of the K neighbors.
- It is a non-parametric method and it does not require any assumptions about the underlying data distribution.
- Therefore, it makes predictions based on the similarity of data points in a given dataset. K-NN is less sensitive to outliers compared to other algorithms.
- It is also called a lazy learner algorithm because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.
- KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.
- ***Note:-*** Choosing an odd value for K is preferred because a state of equality between the two classes would never occur.



- The impact of selecting a smaller or larger K value on the model
  - ✓ *Larger k value:* The case of underfitting occurs when the value of k is increased. In this case, the model would be unable to correctly learn on the training data.
  - ✓ *Smaller k value:* The condition of overfitting occurs when the value of k is smaller. The model will capture all of the training data, including noise. The model will perform poorly for the test data in this scenario.



k-NEAREST NEIGHBOR algorithm:

Training algorithm:

- For each training example *(x, f (x))*, add the example to the list *training_examples*

Classification algorithm:

- Given a query instance x_q to be classified,
  - ✓ Let $x_1 \ldots x_k$ denote the *k* instances from *training_examples* that are nearest to x_q
  - ✓ Return:

$$\hat{f}(x_q) \leftarrow \underset{v \in V}{argmax} \sum_{i=1}^{k} \delta\,(v, f(x_i))$$

where, $\delta(a,b) = 1$ if $a = b$ and , $\delta(a,b) = 0$ otherwise.

Working of KNN algorithm:

- ✓ **Step-1:** Selecting the optimal value of ***K***
- ✓ **Step-2:** Calculate the Euclidean distance of ***K neighbors***
- ✓ **Step-3:** Choose the *K nearest neighbors* as based on Euclidean distance.
- ✓ **Step-4:** Among these K neighbors, voting for Classification based on neighbors count, otherwise calculate the average of neighbors for Regression.
- ✓ **Step-5:** Assign the new data point to the category for which the neighbor count is maximum or in case of regression the calculated average value becomes the predicted output for the target data point.

Advantages of KNN Algorithm:

- ✓ It is simple and easy to implement.
- ✓ Handles both numerical and categorical data.
- ✓ It is robust to the noise and outliers in training data.
- ✓ Adapts easily for newly classified data as it stores in memory for further classification.
- ✓ It requires few hyper parameters i.e., the value of *K*.
- ✓ It can be more effective if the training data is large.

Disadvantages of the KNN Algorithm:

- ✓ Always needs to determine the value of K which may be complex.
- ✓ Computationally expensive for large datasets.
- ✓ KNN algorithm is a Lazy Algorithm and does not scale well. It takes lots of computing power as well as data storage.
- ✓ Sensitive to irrelevant features.
- ✓ Curse of dimensionality which implies the algorithm faces a hard time classifying the data points properly when the dimensionality is too high – leads to ***overfitting***. Hence generally feature selection as well as dimensionality reduction techniques are applied to deal with this problem.

Clustering in Machine Learning

Clustering or cluster analysis is an unsupervised machine learning technique, which groups the unlabelled dataset. It can be defined as *"A way of grouping the data points into different clusters, consisting of similar data points. The objects with the possible similarities remain in a group that has less or no similarities with another group."*

It does it by finding some similar patterns in the unlabelled dataset such as shape, size, color, behavior, etc., and divides them as per the presence and absence of those similar patterns.

After applying this clustering technique, each cluster or group is provided with a cluster-ID. ML system can use this id to simplify the processing of large and complex datasets.

Applications of Clustering:

- ✓ ***Customer Segmentation***: Grouping customers based on behavior or demographics.
- ✓ ***Image Segmentation***: Identifying regions of interest in images.
- ✓ ***Image Compression***: Reducing image size by representing pixels using cluster centroids.
- ✓ ***Document Clustering***: Organizing similar documents (e.g., news articles, research papers).
- ✓ ***Recommendation Systems***: Grouping users or items for personalized recommendations.
- ✓ ***Biological Data Analysis***: Clustering genes, proteins, or cell types.
- ✓ ***Anomaly Detection***: Outliers often form separate clusters.
- ✓ ***Marketing***: It can be used to characterize & discover customer segments for marketing purposes.

Types of Clustering Algorithms:

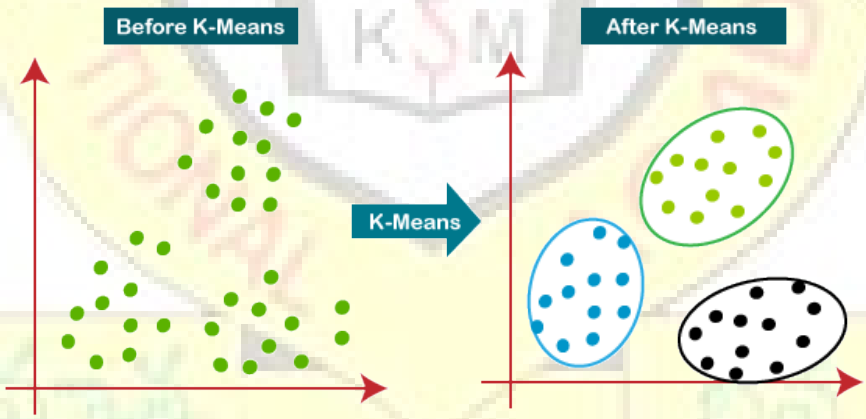
- ✓ ***K-Means Clustering***: Divides data into K clusters based on similarity. It minimizes the sum of squared distances within each cluster.
- ✓ ***Hierarchical Clustering***: Forms a tree-like structure of clusters (dendrogram) based on hierarchy. It can be agglomerative (bottom-up) or divisive (top-down).
- ✓ ***DBSCAN (Density-Based Spatial Clustering of Applications with Noise)***: Identifies clusters based on density and connectivity. It's robust to noise and doesn't require specifying the number of clusters.
- ✓ **Other *Methods***: Gaussian Mixture Models (GMM), Mean-Shift, Spectral Clustering, etc.

Distance based clustering:

- Distance-based clustering is an unsupervised learning technique.
- Distance-based clustering methods are all exemplar-based and hence predictive: they naturally generalise to unseen instances.
- Predictive distance-based clustering methods use the same ingredients as distancebased classifiers: a distance metric, a way to construct exemplars and a distance-based decision rule.

K means Clustering Algorithm

- K-Means Clustering is an Unsupervised Machine Learning algorithm, which groups the unlabeled dataset into different clusters.
- k-Means is one of the popular partition clustering technique, where the data is partitioned into k unique clusters.
- Here K defines the number of pre-defined clusters that need to be created in the process, as if K=2, there will be two clusters, and for K=3, there will be three clusters, and so on.
- *"It is an iterative algorithm that divides the unlabeled dataset into k different clusters in such a way that each dataset belongs only one group that has similar properties."*
- It allows us to cluster the data into different groups and a convenient way to discover the categories of groups in the unlabeled dataset on its own without the need for any training.
- It is a centroid-based algorithm, where each cluster is associated with a centroid.
- The goal of this algorithm is to minimize the sum of distances between the data point and their corresponding clusters.
- The algorithm takes the unlabeled dataset as input, divides the dataset into k-number of clusters, and repeats the process until it does not find the best clusters. The value of k should be predetermined in this algorithm.
- The k-means clustering algorithm mainly performs two tasks:
  - ✓ Determines the best value for K center points or centroids by an iterative process.
  - ✓ Assigns each data point to its closest k-center. Those data points which are near to the particular k-center, create a cluster.
- The K-means problem is NP-complete, which means that there is no efficient solution to find the global minimum and we need to resort to a heuristic algorithm.



How to find the optimal value of K?

- *K* is a hyper-parameter to the K-means Algorithm. In most cases, the number of clusters *K* is determined in a heuristic fashion. The two most popular criteria used are the elbow and the silhouette methods.
  - ✓ The ***Elbow method*** is one of the most popular ways to find the optimal number of clusters. This method uses the concept of WCSS (Within Cluster Sum of Squares) value which defines the total variations within a cluster. To measure the distance between data points and centroid, we can use any method such as Euclidean distance or Manhattan distance.

$$Total\ Within\ Cluster\ Sum\ of\ Squares\ (WCSS) = \frac{1}{N}\sum_{i=1}^{K}\sum_{j=1}^{n_i}\left(x_{ij} - \bar{x}_i\right)^2$$

  - ✓ The ***silhouette algorithm*** is one of the many algorithms to determine the optimal number of

clusters for an unsupervised learning technique.

$$S(x_i) = \frac{b(x_i) - a(x_i)}{\max(a(x_i),\ b(x_i))}$$

Where, $a(x_i)$ = the average distance of x_i to the points in its own cluster.
$b(x_i)$ = the average distance to the points in its neighbouring cluster.

K-means Algorithm: *KMeans(D, K) – K-means clustering using Euclidean distance* Dis_2
Input: data $D \subseteq \mathbb{R}^d$; number of clusters $K \in \mathbb{N}$.
Output: K cluster means $\mu_1,\ldots, \mu_K \in \mathbb{R}^d$.

1. randomly initialize K vectors $\mu_1,\ldots, \mu_K \in \mathbb{R}^d$;
2. repeat
3. assign each $\mathbf{x} \in D$ to $\text{argmin}_j\ Dis_2(x, \mu_j)$;
4. for $j = 1$ to K do
5. $D_j \leftarrow \{\mathbf{x} \in D | \mathbf{x}$ assigned to cluster $j\}$;
6. $\mu_j = \frac{1}{|D_j|}\sum_{x \in D_j} \boldsymbol{x}$;
7. end
8. until no change in $\mu_1,\ldots, \mu_K$;
9. return $\mu_1,\ldots, \mu_K$;

How Does K-Means Work?

Step 1: Choose the number ***K*** to decide the clusters.
Step 2: Select random ***K*** points as centroids (It can be other from the input dataset).
Step 3: Assign each data point to its closest centroid, forming ***K*** predefined clusters.
Step 4: Calculate the variance and place a new centroid for each cluster.
Step 5: Repeat steps 3 and 4, reassigning each data point to the new closest centroid.
Step 6: If reassignment occurs, go back to step 4; otherwise, finish.

Advantages of k-means:

- ✓ **Simplicity**: K-means is relatively simple to implement and understand.
- ✓ **Scalability**: It scales well to large data sets, making it suitable for handling substantial amounts of data.
- ✓ **Convergence Guarantee**: K-means guarantees convergence to a local minimum, ensuring stable results.
- ✓ **Warm-Start Capability**: You can initialize the positions of centroids (cluster centers) from previous runs, improving efficiency.
- ✓ **Adaptability**: K-means adapts well to new examples and can handle clusters of different shapes and sizes (including elliptical clusters).

Disadvantages of k-means:

- ✓ ***Manual Choice of k***: Selecting the optimal number of clusters (k) manually can be challenging.
- ✓ ***Initialization Sensitivity***: K-means is sensitive to initial centroid positions. Running it multiple times with different initializations helps mitigate this issue.
- ✓ ***Varying Cluster Sizes and Densities***: K-means struggles when clusters have different sizes and densities. Generalization techniques can help address this limitation.
- ✓ ***Handling Outliers***: Outliers can affect centroid positions or even form their own clusters. Consider removing or clipping outliers before clustering.
- ✓ ***Dimensionality Scaling***: As the number of dimensions increases, distance-based similarity measures converge to a constant value. Use techniques like PCA or spectral clustering to handle

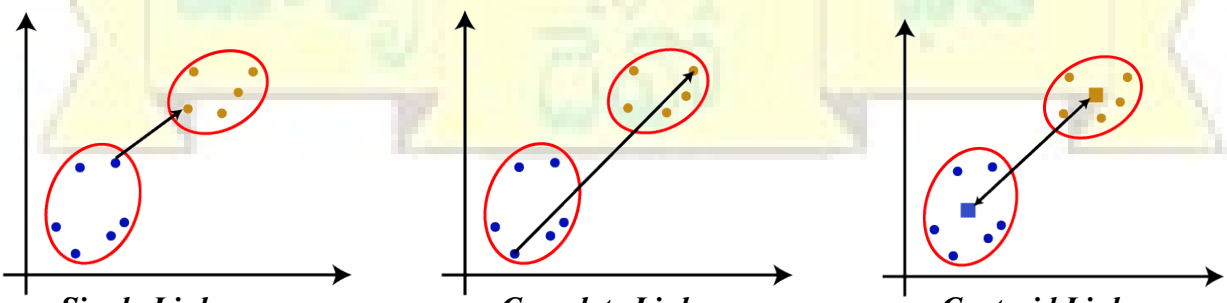
high-dimensional data.

Hierarchical clustering

- Hierarchical clustering is an unsupervised learning technique used to group similar objects into clusters.
- It creates a hierarchy of clusters by merging or splitting them based on similarity measures.
- Hierarchical clustering uses similarities of instances, without any other requirement on the data; the aim is to find groups such that instances in a group are more similar to each other than instances in different groups.
- Hierarchical clustering algorithms are also known as connectivity-based clustering algorithms.
- Here, we develop the hierarchy of clusters in the form of a tree, and this tree-shaped structure is known as the **dendrogram**.
- Generally Euclidean distance is used, where one has to make sure that all attributes have the same scale. This is a special case of the Minkowksi distance with p = 2.
- Sometimes the results of K-means clustering and hierarchical clustering may look similar, but they both differ depending on how they work. As there is no requirement to predetermine the number of clusters as we did in the K-Means algorithm.
- There are two major types of approaches in hierarchical clustering:
  - ✓ Agglomerative Hierarchical Clustering
  - ✓ Divisive Hierarchical clustering

Agglomerative Hierarchical Clustering:

- ✓ An agglomerative clustering algorithm is a ***bottom-up approach*** and starts with N groups, each initially containing one training instance, merging similar groups to form larger groups, until there is a single one.
- ✓ At each iteration, of an agglomerative algorithm, we choose the two closest groups to merge.
- ✓ There are various ways to calculate the distance between two clusters. One among such measures is called **Linkage methods**. The most common linkage functions are as follows::
  - ***Single linkage*** defines the distance between two clusters as the smallest pairwise distance between elements from each cluster.
  - ***Complete linkage*** defines the distance between two clusters as the largest pointwise distance.
  - ***Average linkage*** defines the cluster distance as the average pointwise distance.
  - ***Centroid linkage*** defines the cluster distance as the point distance between the cluster means.



Single Linkage ***Complete Linkage*** ***Centroid Linkage***

These linkage functions can be defined mathematically as follows:

$$L_{single}(A,B) = \min_{x \in A, y \in B} Dis(x,y)$$

$$L_{complete}(A,B) = \max_{x \in A, y \in B} Dis(x,y)$$

$$L_{average}(A,B) = \frac{\sum_{x\in A, y\in B} Dis(x,y)}{|A|.|B|}$$

$$L_{centroid}(A,B) = Dis\left(\frac{\sum_{x\in A} x}{|A|}, \frac{\sum_{y\in B} y}{|B|}\right)$$

Clearly, all these linkage functions coincide for singleton clusters: L({x}, {y}) = Dis(x, y). However, for larger clusters they start to diverge. For example, suppose Dis(x, y) < Dis(x, z), then the linkage between {x} and {y, z} is different in all four cases:

$$L_{single}(\{x\},\{y,z\}) = Dis(x,y)$$

$$L_{complete}(\{x\},\{y,z\}) = Dis(x,z)$$

$$L_{average}(\{x\},\{y,z\}) = (Dis(x,y) + Dis(x,z))/2$$

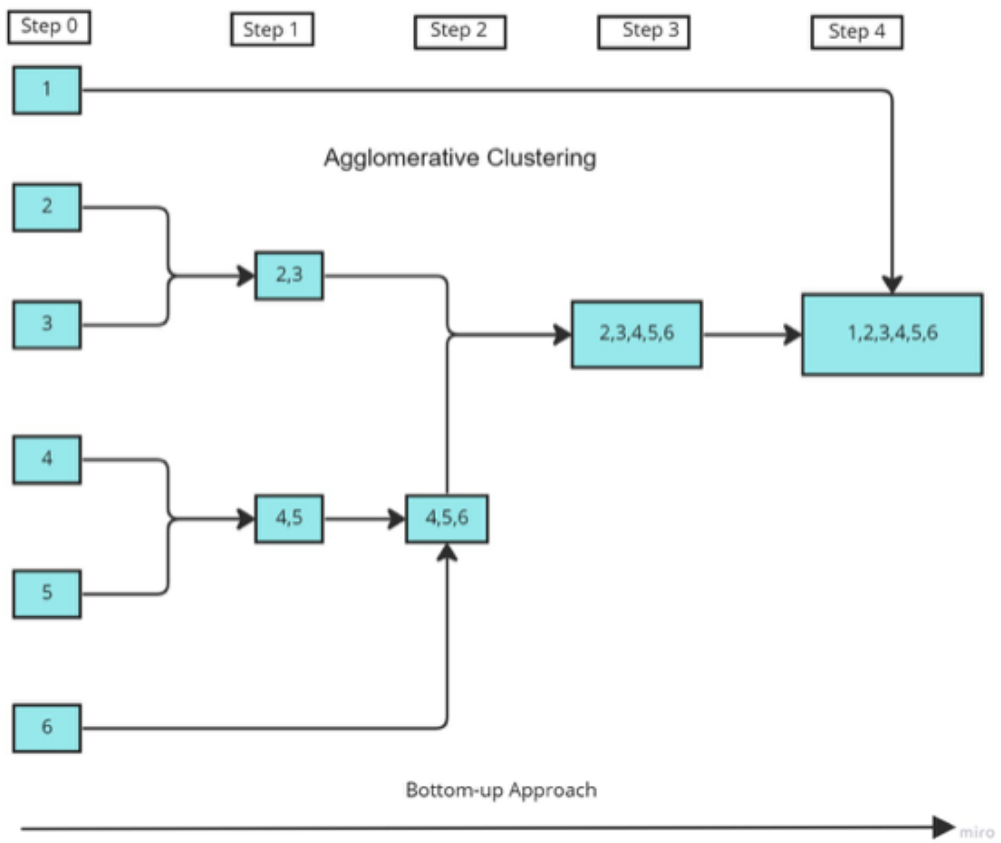
$$L_cetroid\ (\{x\},\{y,z\}) = Dis(x,((y+z))/2)$$

- ✓ Single linkage is the easiest case to understand, as it effectively builds a graph by adding increasingly longer links between points, one at a time.
- ✓ Hierarchical clustering using single linkage can essentially be done by calculating and sorting all pairwise distances between data points, which requires $O(n^2)$ time for n points. The other linkage functions require at least $O(n^2 \log n)$.

Algorithm AHC:-

The steps for agglomerative clustering are as follows:

1. Compute the proximity matrix using a distance metric.
2. Use a linkage function to group objects into a hierarchical cluster tree based on the computed distance matrix from the above step.
3. Data points with close proximity are merged together to form a cluster.
4. Repeat steps 2 and 3 until a single cluster remains.
5. Return the constructed binary tree with linkage levels.



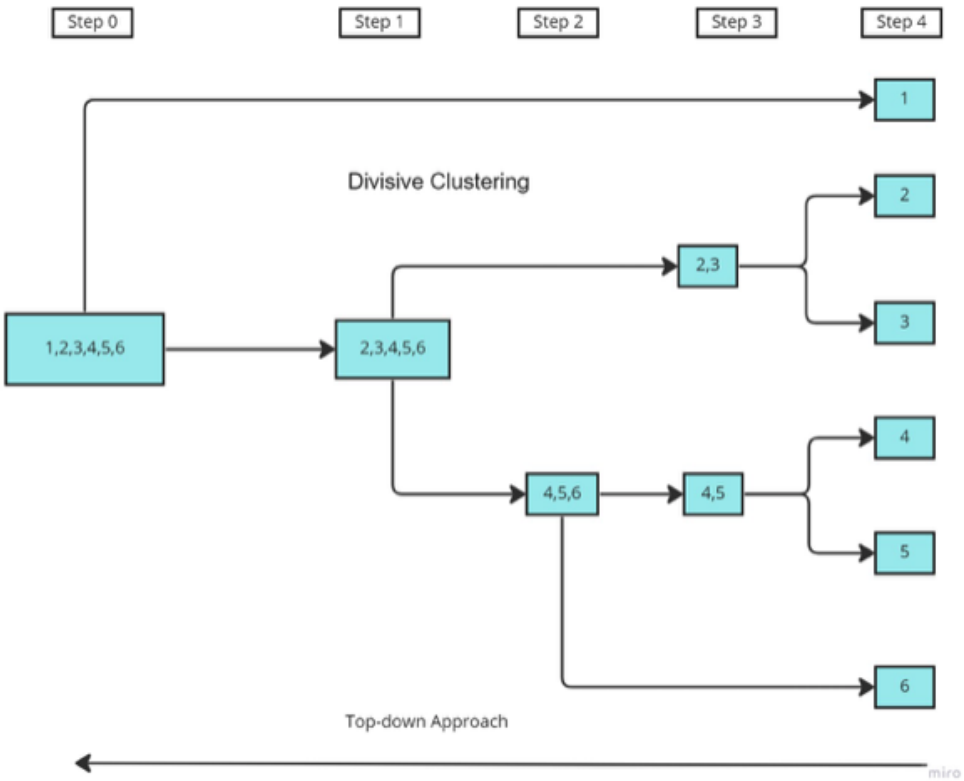
Divisive Hierarchical clustering:

- ✓ A divisive clustering algorithm goes in the other direction ie., ***top-down approach***, starting with a single group and dividing large groups into smaller groups, until each group contains a single instance.
- ✓ It starts by considering all the data points into a big single cluster and later on splitting them into smaller heterogeneous clusters continuously until all data points are in their own cluster. Thus, they are good at identifying large clusters.
- ✓ It follows a top-down approach and is more efficient than agglomerative clustering.
- ✓ But, due to its complexity in implementation, it doesn't have any predefined implementation in any of the major machine learning frameworks.
- ✓ The proximity matrix and linkage function follow the same procedure as agglomerative clustering.

Algorithm DHC:-

The steps for divisive clustering are as follows:

1. Split into clusters using any flat-clustering method, say K-Means.
2. Choose the best cluster among the clusters to split further, choose the one that has the largest Sum of Squared Error (SSE).
3. Repeat steps 2 and 3 until a single cluster is formed.
4. Return the constructed binary tree with linkage levels.



Limits of Hierarchical Clustering

- ✓ It has high time and space computational complexity. For computing proximity matrix, the time complexity is $O(N^2)$, since it takes N steps to search, the total time complexity is $O(N^3)$.
- ✓ There is no objective function for hierarchical clustering.
- ✓ Due to high time complexity, it cannot be used for large datasets.
- ✓ It is sensitive to noise and outliers since we use distance metrics.
- ✓ It has difficulty handling large clusters.

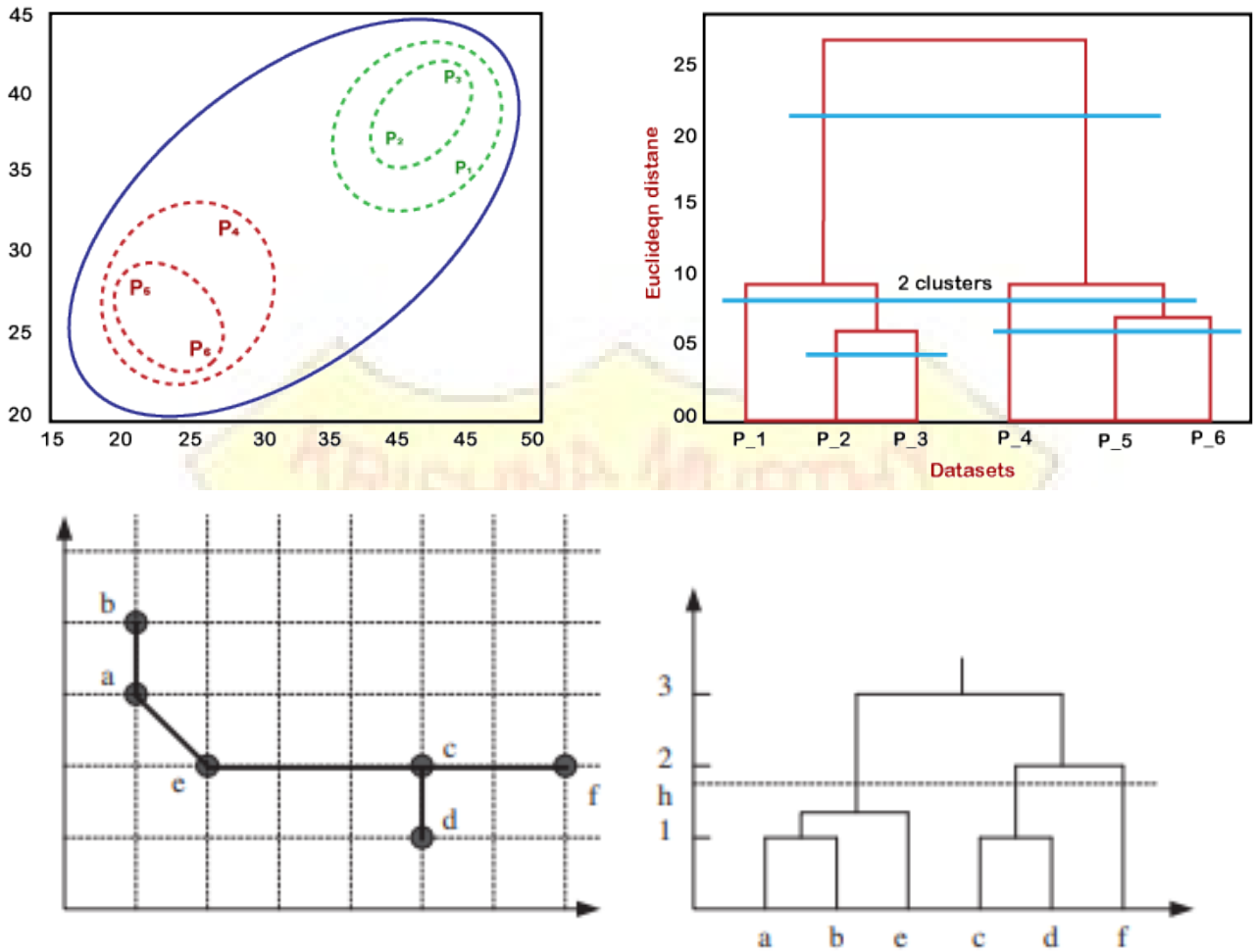


Fig.:- A two-dimensional dataset and the dendrogram, showing the result of single-link clustering. Note that leaves of the tree are ordered so that no branches cross. The tree is then intersected at a desired value of h to get the clusters.

Rule Based Models:

- Rule based models offer more flexibility than tree models: for instance, while decision tree branches are mutually exclusive, the potential overlap of rules may give additional information.
- A rule is a single, independent piece of information. In supervised learning, a rule model is more than just a set of rules: the specification of how the rules are to be combined to form predictions is a crucial part of the model.
- There are essentially two approaches to supervised rule learning. One is inspired by decision tree learning: find a combination of literals – the ***body*** of the rule, which is what we previously called a concept – that covers a sufficiently homogeneous set of examples, and find a label to put in the ***head*** of the rule.
- The second approach goes in the opposite direction: first select a class you want to learn, and then find rule bodies that cover (large subsets of) the examples of that class.
- The first approach naturally leads to a model consisting of an ordered sequence of rules – ***a rule list***.
- The second approach treats collections of rules as unordered ***rule sets***.
- Most rule learning algorithms proceeds as follows: they remove the examples covered by the rule just learned from consideration, and proceed with the remaining examples and is called separate-and-conquer.

- In analogy with the divideand-conquer strategy of decision trees (the difference is that in separate-and-conquer we end up with one remaining sub-problem rather than several as in divide-and-conquer).

Consider small dolphins data set with positive examples:

p1: Length = 3 ∧ Gills = no ∧ Beak = yes ∧ Teeth = many
p2: Length = 4 ∧ Gills = no ∧ Beak = yes ∧ Teeth = many
p3: Length = 3 ∧ Gills = no ∧ Beak = yes ∧ Teeth = few
p4: Length = 5 ∧ Gills = no ∧ Beak = yes ∧ Teeth = many
p5: Length = 5 ∧ Gills = no ∧ Beak = yes ∧ Teeth = few

and negatives

n1: Length = 5 ∧ Gills = yes ∧ Beak = yes ∧ Teeth = many
n2: Length = 4 ∧ Gills = yes ∧ Beak = yes ∧ Teeth = many
n3: Length = 5 ∧ Gills = yes ∧ Beak = no ∧ Teeth = many
n4: Length = 4 ∧ Gills = yes ∧ Beak = no ∧ Teeth = many
n5: Length = 4 ∧ Gills = no ∧ Beak = yes ∧ Teeth = few

Rule learning for subgroup discovery

- When learning classification models it is natural to look for rules that identify pure subsets of the training examples: i.e., sets of examples that are all of the same class and that all satisfy the same conjunctive concept.
- Subgroup discovery focuses on finding ***meaningful patterns*** within a dataset.
- Subgroup discovery provides interpretable rules that describe specific subsets of data.
- Instead of creating a global model, it seeks to discover ***local patterns*** that apply to specific subgroups of instances.
- These subgroups can be defined by certain conditions or characteristics.
- These rules typically involve ***if-then*** statements based on attribute values.
- Subgroup discovery is useful in various domains, such as healthcare, marketing, and social sciences.
- A good subgroup is one whose class distribution is significantly different from the overall population.
- For instance, one could argue that the complement of a subgroup is as interesting as the subgroup itself: in our dolphin example, the concept Gills = yes, which covers four negatives and no positives, could be considered as interesting as its complement Gills = no, which covers one negative and all positives. This means that we need to move away from impurity-based evaluation measures.
- The precision (Laplace-corrected precision $prec^L$) of any subgroup on the diagonal is equal to the proportion of positives, so this leads to $|prec - pos|$ ($|prec^L - pos|$) as one possible quality measure.
- Subgroups on the diagonal always have average recall 0.5, regardless of the class distribution.
- So, a good subgroup evaluation measure is $|avg\text{-}rec - 0.5|$. Average recall can be written as $(1 + tpr - fpr)/2$, and thus $|avg\text{-}rec - 0.5| = |tpr - fpr|/2$.
- A related subgroup evaluation measure is weighted relative accuracy, which can be written as ***pos·neg(tpr −fpr)***.

Subgroup	*Coverage*	$prec^L$	*Rank*	*avg-rec*	*Rank*
Gills = yes	[0+,4−]	0.17	1	0.10	1–2
Gills = no ∧ Teeth = many	[3+,0−]	0.80	2	0.80	3
Gills = no	[5+,1−]	0.75	3–9	0.90	1–2
Beak = no	[0+,2−]	0.25	3–9	0.30	4–11
Gills = yes ∧ Beak = yes	[0+,2−]	0.25	3–9	0.30	4–11
Length = 3	[2+,0−]	0.75	3–9	0.70	4–11
Length = 4 ∧ Gills = yes	[0+,2−]	0.25	3–9	0.30	4–11
Length = 5 ∧ Gills = no	[2+,0−]	0.75	3–9	0.70	4–11
Length = 5 ∧ Gills = yes	[0+,2−]	0.25	3–9	0.30	4–11
Length = 4	[1+,3−]	0.33	10	0.30	4–11
Beak = yes	[5+,3−]	0.60	11	0.70	4–11

***Table*:-** Detailed evaluation of the top subgroups. Using Laplace-corrected precision we can evaluate the quality of a subgroup as $|prec^L - pos|$. Alternatively, we can use average recall to define the quality of a subgroup as $|avg\text{-}rec - 0.5|$. These two quality measures result in slightly different rankings.

Association rule mining

- Association rule mining is a type of unsupervised learning technique finds interesting associations and relationships/patterns among large sets of data items. It shows how frequently a itemset occurs in a transaction.
- It is based on different rules to discover the interesting relations between variables in the database.
- It is employed in **Market Basket analysis, Symptoms of any disease, Fraud Detection, Web usage mining, Recommendation systems, etc**.
- Market Based Analysis is one of the key techniques used by large relations to show associations between items. It allows retailers to identify relationships between the items that people buy together frequently. Based on this the retailer identifies the best product placement in the store and deciding offers which increases the overall sales.

How does Association Rule mining work?

Association rule mining is based on ***if-then*** rules, such as if A then B.

Here, the ***if*** element is called **antecedent**, and ***then*** statement is called as **Consequent**.

- In these types of relationships, we can find out some association or relation between two items is known *as single cardinality*. It is all about creating rules, and if the number of items increases, then cardinality also increases accordingly. So, to measure the associations between thousands of data items, there are several metrics. These metrics are given below:
  - ✓ Support
  - ✓ Confidence
  - ✓ Lift

Support:

Support is a measure of how frequently an item or itemset appears in the dataset. It is calculated as the number of transactions containing the item(s) divided by the total number of transactions in the dataset. High support indicates that an item or itemset is common in the dataset, while low support indicates that it is rare.

$$Support(\{X\} \rightarrow \{Y\}) = \frac{Transactions\ containing\ both\ X\ and\ Y}{Total\ number\ of\ transactions}$$

Confidence

Confidence is a measure of the strength of the association between two items. It is calculated as the number of transactions containing both items divided by the number of transactions containing the first item. High confidence indicates that the presence of the first item is a strong predictor of the presence of the second item.

$$Confidence(\{X\} \rightarrow \{Y\}) = \frac{Transactions\ containing\ both\ X\ and\ Y}{Transactions\ containing\ X}$$

Lift

Lift is a measure of the strength of the association between two items, taking into account the frequency of both items in the dataset. It is calculated as the confidence of the association divided by the support of the second item. Lift is used to compare the strength of the association between two items to the expected strength of the association if the items were independent.

A value of lift >1 (<1) indicates that the association between two items is stronger (weaker) than expected based on the frequency of the individual items. If lift = 1 then the items are independent.

$$Lift(\{X\} \rightarrow \{Y\}) = \frac{(Transactions\ containing\ both\ X\ and\ Y)/(Transactions\ containing\ X)}{Fraction\ of\ transactions\ containing\ Y}$$

$$Lift(\{X\} \rightarrow \{Y\}) = \frac{Confidence(\{X\} \rightarrow \{Y\})}{Expected\ Confidence}, where\ Expectected\ Confidence = Support(Y)$$

Association Rule Mining Algorithms

There are several algorithms used for association rule mining. Some common ones are:

Apriori algorithm

The Apriori algorithm is one of the most widely used algorithms for association rule mining. It works by first identifying the frequent itemsets in the dataset (itemsets that appear in a certain number of transactions). It then uses these frequent itemsets to generate association rules, which are statements of the form "if item A is purchased, then item B is also likely to be purchased." The Apriori algorithm uses a bottom-up approach, starting with individual items and gradually building up to more complex itemsets.

ECLAT algorithm

The ECLAT (Equivalence Class Transformation and bottom-up Lattice Traversal) algorithm is a variation of the Apriori algorithm that uses a top-down approach rather than a bottom-up approach. It works by dividing the items into equivalence classes based on their support (the number of transactions in which they appear). The association rules are then generated by combining these equivalence classes in a lattice-like structure. It is a more efficient and scalable version of the Apriori algorithm.

FP-Growth algorithm

The FP-Growth (Frequent Pattern Growth) algorithm is another popular algorithm for association rule mining. It works by constructing a tree-like structure called a FP-tree, which encodes

the frequent itemsets in the dataset. The FP-tree is then used to generate association rules in a similar manner to the Apriori algorithm. The FP-Growth algorithm is generally faster than the Apriori algorithm, especially for large datasets.

Example:-

Given a set of transactions, we can find rules that will predict the occurrence of an item based on the occurrences of other items in the transaction.

TID	Items
1	Bread, Milk
2	Bread, Diaper, Coke, Eggs
3	Milk, Diaper, Coke, Cake
4	Bread, Milk, Diaper, Coke
5	Bread, Milk, Diaper, Cake

From above table, {Milk, Diaper}=>{Cake}

Note, Here X = {Milk, Diaper} and Y= {Cake}

Support (s) $= \frac{\sigma\,(\{Milk,Diaper,Crisps\})}{Total\ Transactions} = \frac{2}{5} = 0.4 = 40\%$

Confidence (c) $= \frac{\sigma\,(\{Milk,Diaper,Crisps\})}{\sigma\,(\{Milk,Diaper\})} = \frac{2}{3} = 0.67 = 67\%$

Lift (l) $= \frac{Support\,(\{Milk,Diaper,Crisps\})}{Support\,(\{Milk,Diaper\}) * Support(\{Crisps\})} = \frac{0.4}{(0.6 * 0.6)} = 1.11$

Tree Based Models:

- **Tree-based models** are widely used for both classification and regression tasks.
- They work by recursively breaking down a dataset into smaller subsets while simultaneously creating a tree-like structure of decisions that ultimately leads to predictions.
- Trees are expressive and easy to understand due to their recursive 'divide-and-conquer' nature.
- Divide-and-conquer algorithm divides the data into subsets, builds a tree for each of those and then combines those subtrees into a single tree.
- As, logical rules are easily arranged in a tree structure, which we refer to as a ***feature tree***.
- A ***feature tree*** is a tree such that each internal node (the nodes that are not leaves) is labelled with a feature, and each edge emanating from an internal node is labelled with a literal. The set of literals at a node is called a split. Each leaf of the tree represents a logical expression, which is the conjunction of literals encountered on the path from the root of the tree to the leaf. The extension of that conjunction (the set of instances covered by it) is called the instance space segment associated with the leaf.
- Feature trees whose leaves are labelled with classes are commonly called ***decision trees***.
- Tree-learning algorithms typically works in a top–down fashion are sometimes called a separate-and-conquer approach.
- Note, Decision trees are strictly more expressive than conjunctive concepts.
- Divide and conquer algorithms are greedy: whenever there is a choice (such as choosing the best split), the best alternative is selected on the basis of the information then available, and this choice is never reconsidered. This may lead to sub-optimal choices. An alternative would be to use a ***backtracking search algorithm***, which can return an optimal solution, at the expense of increased computation time and memory requirements.

- Some major tree-based algorithms are **Decision Tree**, **Random Forest, XGBoost, Ranking and Probability estimation Trees, Regression trees, Clustering Trees**.

How do Tree-based algorithms work?

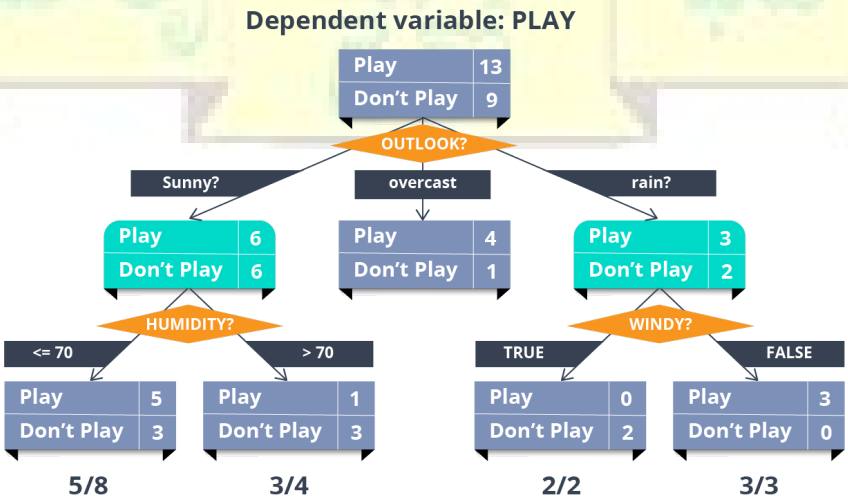
The main four workflows of tree-based algorithms are discussed below:

1. ***Feature Splitting*:** Tree-based algorithms begin by selecting more informative features to split a data set based on a specific criterion, such as Gini impurity or information gain etc.
2. ***Recursive splitting*:** The selected feature of dataset is used to split the data in two, and the process is repeated for each resulting subset, forming a hierarchical binary tree structure. This recursive splitting until stops a predefined criterion, like a maximum depth or a minimum number of samples per train data, is met as long as it lasts.
3. ***Leaf Node Function*:** As the tree grows, each terminal node (leaf) is given a predicted outcome based on majority learning (for classification) or the sample value of that node of the (for regression). This activates the tree to capture complex decision boundaries and relationships in the data.
4. ***Ensemble Learning*:** For ensemble methods like Random Forests and Gradient Boosting Machines, multiple trees are trained independently, and their predictions are combined to obtain the final result. This group approach helps to reduce overfitting, increase generalization, and improve overall model performance by combining the strengths of individual trees and reducing their weaknesses.

- Note, the splitting process involves Attribute selection measures such as Entropy and Gini Index.

Decision Trees

- A **Decision Tree** is a supervised learning technique commonly used for both **classification** and **regression** tasks.
- A decision tree can handle both categorical data (YES/NO) and numerical data.
- Decision Trees are simple to understand, interpret, and visualize.
- It is a tree-structured classifier, where internal nodes represent the features of a dataset, branches represent the decision rules and each leaf node represents a prediction or a class label.
- A decision tree simply based on logical rules, i.e., (Yes/No), it further split the tree into subtrees.
- Decision Tree algorithm ***recursively partitions the data*** based on the values of features and selects the best-split criteria to minimize the impurity or maximize the information gain.
- Decision trees can handle high-dimensional data and does not require any domain knowledge or parameter setting.
- In order to build a decision tree, we use the ***CART (Classification and Regression Tree) algorithm.***



Decision Tree Terminologies

There are specialized terms associated with decision trees that denote various components and facets of the tree structure and decision-making procedure:

- ✓ ***Root Node***: A decision tree's root node, which represents the original choice or feature from which the tree branches, is the highest node.
- ✓ ***Internal Nodes (Decision Nodes)***: Nodes in the tree whose choices are determined by the values of particular attributes. There are branches on these nodes that go to other nodes.
- ✓ ***Leaf Nodes (Terminal Nodes)***: The branches' termini, when choices or forecasts are decided upon. There are no more branches on leaf nodes.
- ✓ ***Branches (Edges)***: Links between nodes that show how decisions are made in response to particular circumstances.
- ✓ ***Splitting***: The process of dividing a node into two or more sub-nodes based on a decision criterion. It involves selecting a feature and a threshold to create subsets of data.
- ✓ ***Parent Node***: A node that is split into child nodes. The original node from which a split originates.
- ✓ ***Child Node***: Nodes created as a result of a split from a parent node.
- ✓ ***Decision Criterion***: The rule or condition used to determine how the data should be split at a decision node. It involves comparing feature values against a threshold.
- ✓ ***Pruning***: The process of removing branches or nodes from a decision tree to improve its generalization and prevent overfitting.
- ✓ ***Impurity***: A measurement of the target variable's homogeneity in a subset of data. It refers to the degree of randomness or uncertainty in a set of examples. The **Gini index** and **entropy** are two commonly used impurity measurements in decision trees for classifications task
- ✓ ***Variance***: Variance measures how much the predicted and the target variables vary in different samples of a dataset. It is used for regression problems in decision trees. **Mean squared error** and **Mean Absolute Error** are used to measure the variance for the regression tasks in the decision tree.
- ✓ **Information Gain:** Information gain is a measure of the reduction in impurity achieved by splitting a dataset on a particular feature in a decision tree. The splitting criterion is determined by the feature that offers the greatest information gain, It is used to determine the most informative feature to split on at each node of the tree, with the goal of creating pure subsets

Decision Tree Algorithm:-

Step-1: Begin the tree with the root node, says S, which contains the complete dataset.

Step-2: Find the best attribute in the dataset using **Attribute Selection Measure (ASM).**

Step-3: Divide the S into subsets that contains possible values for the best attributes.

Step-4: Generate the decision tree node, which contains the best attribute.

Step-5: Recursively make new decision trees using the subsets of the dataset created in step-3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

Attribute Selection Measures:

- ✓ Attribute selection measure (ASM) is a criterion used in decision tree algorithms to evaluate the usefulness of different attributes for splitting a dataset.
- ✓ The goal of ASM is to identify the attribute that will create the most homogeneous subsets of data after the split, thereby maximizing the information gain.
- ✓ Two popular Attribute selection measures for splitting are: ***Gini Index*** and ***Entropy.***

Gini Impurity (Gini Index)

- ✓ Gini impurity is a metric used to measure the impurity or disorder within a dataset.
- ✓ It is commonly employed in decision tree algorithms, such as the ***Classification and Regression Tree*** **(CART)** algorithm, to decide the best splitting points for nodes.
- ✓ The Gini impurity ranges from 0 (perfectly pure, where all instances belong to the same class) to 1 (completely impure, where instances are evenly distributed across classes).
- ✓ When constructing a decision tree, the algorithm evaluates different features to determine the best split.
- ✓ For each feature, it calculates the Gini impurity after the split and selects the feature that minimizes the impurity.
- ✓ The chosen feature becomes the splitting criterion for that node, leading to the creation of child nodes.
- ✓ The Gini impurity for a given node can be calculated using the following formula:

$$Gini(D) = 1 - \sum_{i=1}^{k} (p_i)^2$$

Where, D represents the dataset at the node.

k is the number of classes.

p_i is the proportion of instances belonging to class i in the node.

Entropy

- ✓ Entropy is a measure of information uncertainty in a dataset.
- ✓ In the context of decision trees, it quantifies the impurity or disorder within a node.
- ✓ The entropy value always lies between 0 and 1, but depending on the number of classes in the dataset, it can exceed 1.
- ✓ The splitting process involves assessing candidate splits based on the reduction in entropy they induce.
- ✓ The algorithm selects the split that maximizes the information gain, representing the reduction in uncertainty achieved by the split.
- ✓ This results in nodes with more ordered and homogenous class distributions, contributing to the overall predictive power of the tree.
- ✓ The entropy can be calculated using the following formula:

$$Entropy\ (P) = -\sum_{i=1}^{N} p_i \log_2 p_i$$

Where,

p_i - represents the probability of randomly selecting an example from class i.

$\log_2 p_i$ – represents the uncertainty or the impurity

i = 2 means our problem is a binary classification.

Information Gain

- ✓ Information gain is a concept derived from entropy, measuring the reduction in uncertainty about the outcome variable achieved by splitting a dataset based on a particular feature.
- ✓ In tree-based algorithms, the splitting process involves selecting the feature and split point that maximize information gain.
- ✓ High information gain implies that the split effectively organizes and separates instances, resulting in more homogeneous subsets with respect to the target variable.

- ✓ The goal is to iteratively choose splits that collectively lead to a tree structure capable of making accurate predictions on unseen data.
- ✓ Information Gain can be calculated as follows:

$$IG(S,a) = H(S) - H(S|a)$$

Where,

IG(S, a) - is the information for the dataset *S* for the variable *'a'* for a random variable.

H(S) - is the entropy for the dataset before any change

H(S | a) - is the conditional entropy for the dataset given the variable *a*.

- ✓ Let's suppose that the attribute a can take m values: $a_1, a_2, \ldots, a_m$ and that p_i is the fraction of the objects in S with $a = a_i$. Then, the **information gain** of a_i is:

$$IG(S,a) = H(S) - \sum_{i=1}^{m} p_i H(S|a = a_i)$$

Advantages of Decision Tree

- ✓ Simple and easy to understand, implement and generates understandable rules.
- ✓ Handle both numerical (regression) and categorical (classification) data without requiring extensive preprocessing.
- ✓ Provides insights into feature importance for decision-making.
- ✓ Handle missing values and outliers without significant impact.

Disadvantages of Decision Tree

- ✓ The decision tree contains lots of layers, which makes it complex.
- ✓ Potential for overfitting, which can be resolved using the Random Forest algorithm.
- ✓ Sensitivity to small changes in data, limited generalization if training data is not representative.
- ✓ ***Pruning algorithms*** can also be expensive since many candidate sub-trees must be formed and compared.

***Note*:- Pruning** is a process of deleting the unnecessary nodes from a tree in order to get the optimal decision tree.

Example to have more insights on Decision Trees:-

R_ID	Age	Income	Student	Credit_rating	Buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Table: Class-labeled training tuples from AllElectronics customer database.

We will use the **Attribute Selection Measure** at each node to split the dataset to find the best tree.
Let,

D: The given data partition

A: Attribute set

P_i: the probability that a target feature takes.

Considering the above table:

Entropy: ***(=0** means pure, **=1** impure)*

Entropy(D) = Entropy (Buys_computer)

$= -(9/14*\log_2(9/14) - 5/14*\log_2(5/14)) =$ **0.940** (means the dataset is impure)

Gain(D, Age) $= H(D) - H(D|Age)$

$= \text{Entropy(D)} - (5/14) * \text{Entropy}(S_{youth}) - (4/14) * \text{Entropy}(S_{middle_aged}) - (5/14) * \text{Entropy}(S_{senior})$

$= 0.940 - (5/14)*(0.971) - (4/14)*(0.0) - (5/14)*(0.971)$

$= 0.940 - 0.694$

$=$ **0.246**

Similarily,

Gain(D, Income) = **0.029**

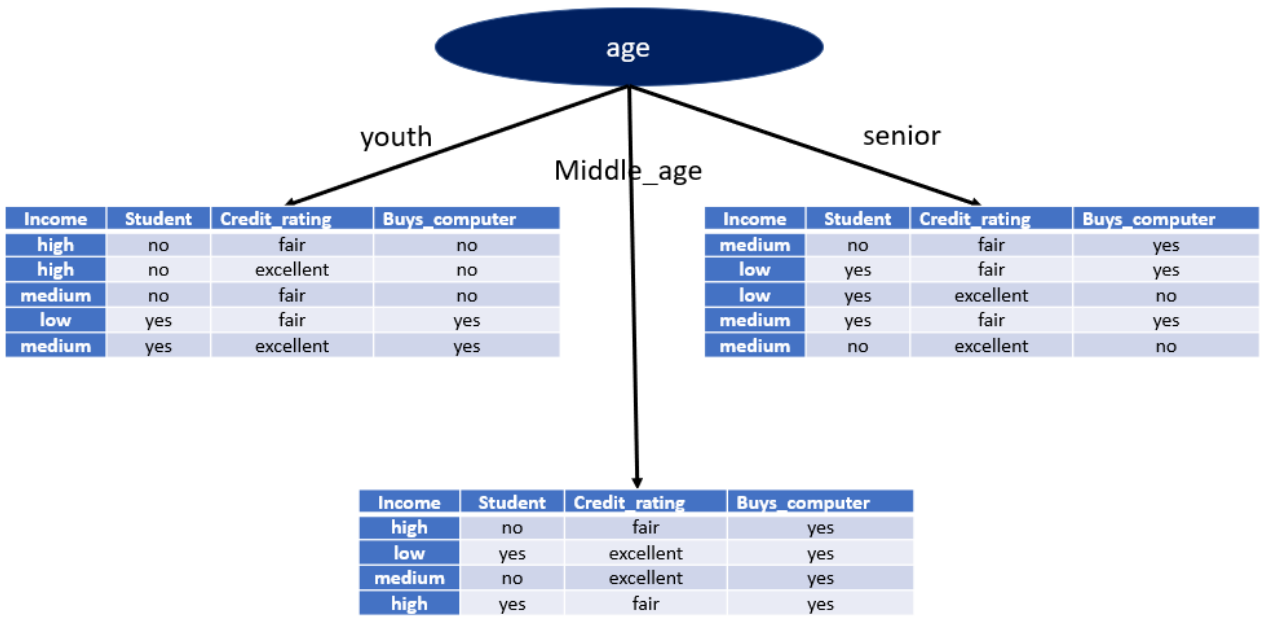
Gain(D, Student) = **0.151**

Gain(D, Credit_rating) = **0.048**

On comparing these values of gain for all the attributes, we find out that the 'information gain' for 'Age' is the highest. Thus, splitting at 'age' is a good decision.

Similarly, at each split, we compare the information gain to find out whether that attribute should be chosen for split or not.

Thus, the sub-optimal tree created looks like:



Income	Student	Credit_rating	Buys_computer
high	no	fair	no
high	no	excellent	no
medium	no	fair	no
low	yes	fair	yes
medium	yes	excellent	yes

Income	Student	Credit_rating	Buys_computer
medium	no	fair	yes
low	yes	fair	yes
low	yes	excellent	no
medium	yes	fair	yes
medium	no	excellent	no

Income	Student	Credit_rating	Buys_computer
high	no	fair	yes
low	yes	excellent	yes
medium	no	excellent	yes
high	yes	fair	yes

Gini Index: ***(**attribute with low Gini index is preferred**)***

Let's calculate the Gini Index for the credit_rating:

P(credit_rating = fair) = 8/14

P(credit_rating = excellent) = 6/14

P(credit_rating = fair & Buys_computer = yes) = 6/8

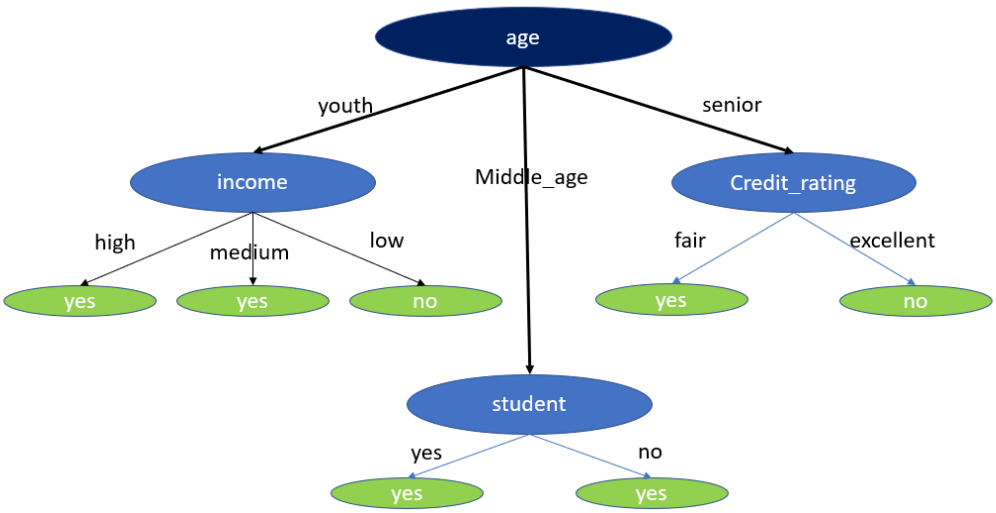
P(credit_rating = fair & Buys_computer = no) = 2/8

P(credit_rating = excellent & Buys_computer = yes) = 3/6
P(credit_rating = excellent & Buys_computer = no) = 3/6
Gini = $1 - ((6/8)^2 + (2/8)^2) = 0.375$
Gini = $1 - ((3/6)^2 + (3/6)^2) = 0.50$
Weighted Gini(Credit_rating) = (8/14)*(0.375) + (6/14)*(0.50) = **0.428**

Similarly,

Weighted Gini(Age) = **0.343**
Weighted Gini(Income) = **0.440**
Weighted Gini(Student) = **0.368**

Recursively, we obtain the optimal tree as follows:



Ranking and Probability estimation Trees

- Grouping classifiers such as decision trees divide the instance space into segments, and so can be turned into rankers by learning an ordering on those segments.
- Unlike some other grouping models, decision trees have access to the local class distributions in the segments or leaves, which can directly be used to construct a leaf ordering that is optimal for the training data.
- Unlike regular decision trees that provide discrete class predictions, Probability estimation Trees assign probabilities to each class.
- Probability estimation Trees are particularly useful when we want not only the predicted class but also the **likelihood** of an example belonging to that class. ***Example:*** Direct marketing of a product.
- Continuous attributes can be handled by sorting examples based on attribute values and choosing midpoints between consecutive values for splits.
- Larger trees may be better for probability estimation, even if they are not necessary for accuracy maximization.
- To turn a feature tree into a ranker, we order its leaves on non-increasing empirical probabilities, which is provably optimal on the training set.
- To turn the tree into a probability estimator, we predict the empirical probabilities in each leaf, applying Laplace or *m*-estimate smoothing to make these estimates more robust for small leaves.
- To turn the tree into a classifier, we choose the operating conditions and find the operating point that is optimal under those operating conditions.

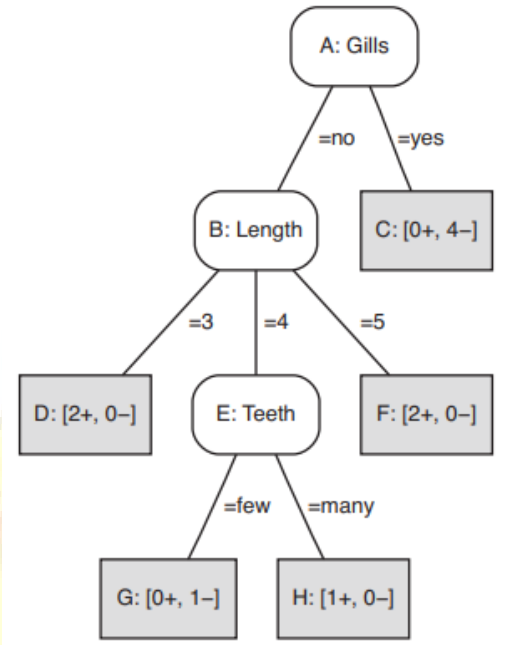


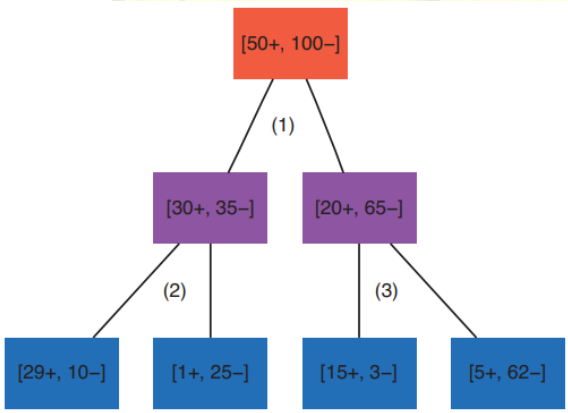
Fig:- Decision tree learned from the data in above example

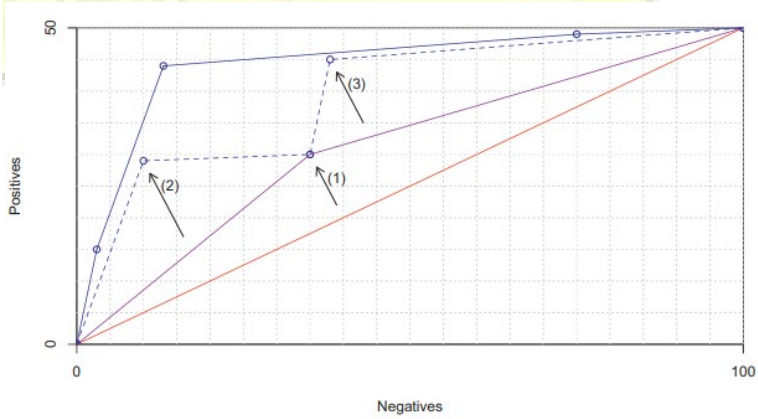
- So, for instance, in Figure above this ordering is [D – F] – H – G – C, resulting in a perfect ranking (AUC = 1).
- The ordering can simply be obtained from the empirical probabilities $\dot{p}$, breaking ties as much as possible by giving precedence to leaves covering a larger number of positives.
- The ranking obtained from the empirical probabilities in the leaves of a decision tree yields a convex ROC curve on the training data.

Example:-

Consider the tree in figure below: Each node is labelled with the numbers of positive and negative examples covered by it: so, for instance, the root of the tree is labelled with the overall class distribution (50 positives and 100 negatives), resulting in the trivial ranking [50+, 100−]. The corresponding one-segment coverage curve is the ascending diagonal (See figure below (right)).

- ✓ Adding split (1) refines this ranking into [30+, 35−][20+, 65−], resulting in a two-segment curve.
- ✓ Adding splits (2) and (3) again breaks up the segment corresponding to the parent into two segments corresponding to the children.
- ✓ However, the ranking produced by the full tree – [15+, 3−][29+, 10−][5+, 62−][1+, 25−] – is different from the left-to-right ordering of its leaves, hence we need to reorder the segments of the coverage curve, leading to the top-most, solid curve. This reordering always leads to a convex coverage curve.





So, adding a split to a decision tree can be interpreted in terms of coverage curves as the following two-step process:

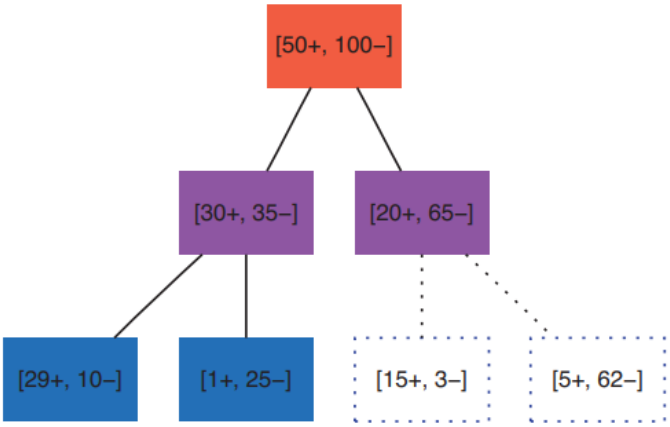
- ✓ split the corresponding curve segment into two or more segments;
- ✓ reorder the segments on decreasing slope.

In general, if a feature tree has l leaves and we have c classes, then the number of possible leaf labellings with classes is c^l ; in the example of Figure above this is $2^4 = 16$.

Assume the training set class ratio clr = 50/100 is representative. We have a choice of five labellings, depending on the expected cost ratio $c = c_{FN}/c_{FP}$ of misclassifying a positive in proportion to the cost of misclassifying a negative:

- ✓ + − + − would be the labelling of choice if c = 1, or more generally if 10/29 < c < 62/5;
- ✓ + − + + would be chosen if 62/5 < c < 25/1;
- ✓ + + + + would be chosen if 25/1 < c; i.e., we would always predict positive if false negatives are more than 25 times as costly as false positives, because then even predicting positive in the second leaf would reduce cost;
- ✓ − − + − would be chosen if 3/15 < c < 10/29;
- ✓ − − − − would be chosen if c < 3/15; i.e., we would always predict negative if false positives are more than 5 times as costly as false negatives, because then even predicting negative in the third leaf would reduce cost.

So let's assume that the class distribution is representative and that false negatives (e.g., not diagnosing a disease in a patient) are about 20 times more costly than false positives.

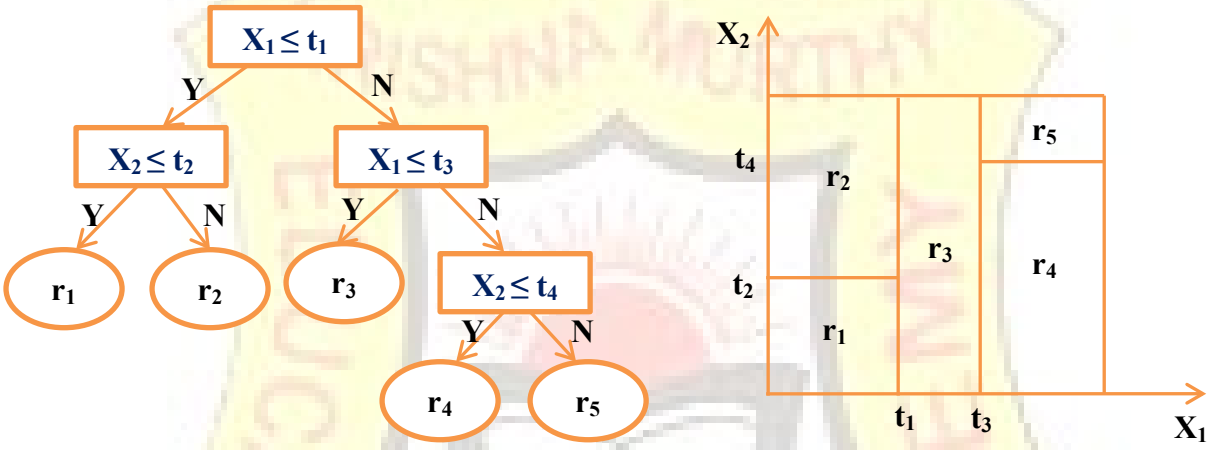


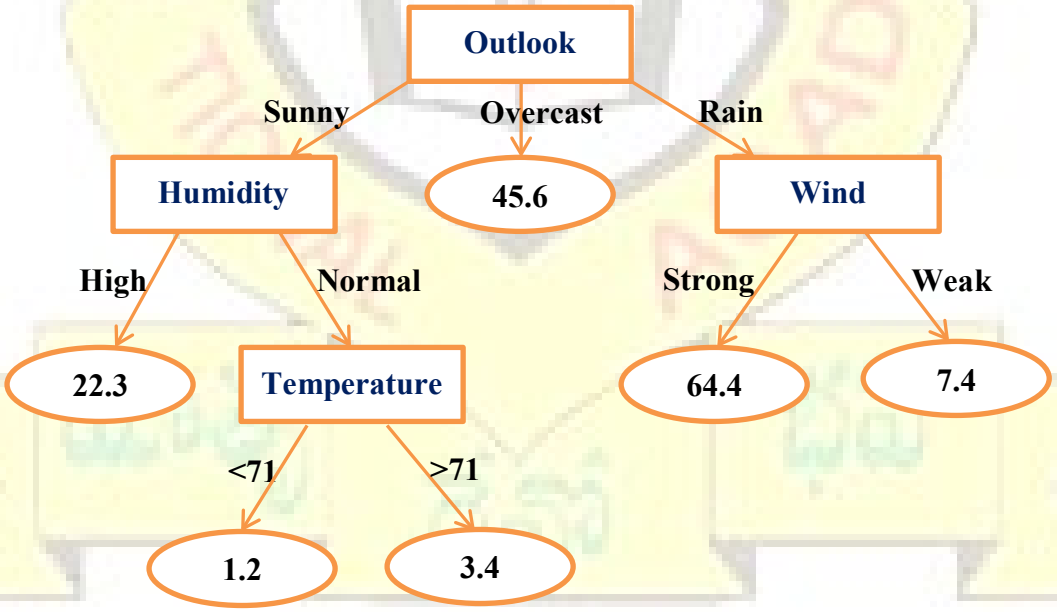
Hence, to achieve the labelling + − + + we don't need the right-most split, which can therefore be pruned away.

- Note, the operation of merging all leaves in a subtree is called pruning the subtree.
- Pruning doesn't affect the chosen operating point, but it does decrease the ranking performance of the tree.
- Pruning is therefore not recommended unless
  - i. you only intend to use the tree for classification, not for ranking or probability estimation; and
  - ii. you can define the expected operating conditions with sufficient precision.
- The popular algorithms for pruning decision trees are Reduced-Error Pruning and Cost Complexity Pruning.
- Note, a too-large tree increases the risk of overfitting, and a small tree may not capture all the important features of the dataset.

Regression trees

- A regression tree is basically a decision tree that is used for the task of regression which can be used to predict continuous valued outputs instead of discrete outputs. For example, if the response variable is the temperature of the day.
- The leaf nodes of a Regression Tree represent the predicted values, and the objective is to minimize the sum of squared errors between the predicted values and actual values.
- **CART** *(Classification And Regression Trees)* is a variation of the decision tree algorithm. It can handle both classification and regression tasks.
- Each node in a regression tree represents a **decision point**, and the branches represent the **possible outcomes** of those decisions.
- The **leaf nodes** contain predicted **numeric values** for the target variable.





How Does CART works for Regression?

- ✓ Regression CART works by splitting the training data recursively into smaller subsets based on specific criteria.
- ✓ The objective is to split the data in a way that minimizes the residual reduction in each subset.
  - ❖ **Residual Reduction-** Residual reduction is a measure of how much the average squared difference between the predicted values and the actual values for the target variable is reduced by splitting the subset. The lower the residual reduction, the better the model fits the data.

- **Splitting Criteria-** Regression trees use a **greedy approach** to split the data at each node with greatest reduction of residual error for the attribute/feature in the resulting subsets. This process is repeated until a stopping criterion is met, such as reaching the maximum tree depth or having too few instances in a leaf node.

Mean Square Error

- In regression problems the target variable is continuous rather than binary and in that case we can define the variance rather than calculating the entropy and gini index.
- Therefore, we try to reduce the Mean Square Error at each child rather than the entropy.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- So minimizing variance over all possible splits of a given parent is the same as maximizing the weighted average of squared means in the children.

Applications of Regression Trees:

- ✓ Predicting house prices based on features like area, location, and amenities.
- ✓ Estimating sales revenue based on advertising expenditure.
- ✓ Forecasting stock prices using historical data.

Advantages:

- ✓ Regression trees are simple, they are Nonparametric and Nonlinear.
- ✓ Outliers have no meaningful effect on regression trees.
- ✓ Regression trees implicitly perform feature selection.

Disadvantages:

- ✓ Regression tree algorithm is prone to overfitting. So, It is better to always specify the minimum number of children per leaf node in advance and use cross-validation to find this Value.
- ✓ The tree structure may be unstable.

Clustering Trees

- The simple kind of regression tree considered here also suggests a way to learn clustering trees.
- It is perhaps surprising, since regression is a supervised learning problem while clustering is unsupervised.
- The key insight is that regression trees find instance space segments whose target values are tightly clustered around the mean value in the segment – indeed, the variance of a set of target values is simply the (univariate) average squared Euclidean distance to the mean.
- An immediate generalization is to use a vector of target values.
- These trees help us see how the clustering results change when we vary the number of clusters.
- Clustering techniques are commonly employed in data analysis to group similar samples based on shared properties.
- However, determining the optimal number of clusters can be challenging.
- Clustering trees address this challenge by showing how clusters evolve as we adjust the resolution (i.e., the number of clusters).

How Do Clustering Trees Work?

- Imagine a tree structure where each node represents a cluster.
- As we move up the tree, clusters merge to form larger ones.
- The leaves of the tree correspond to individual samples, and the root represents the entire dataset.
- By cutting the tree at different levels, we obtain different clustering's.

Applications and Examples:

- ✓ Single-Cell RNA-Sequencing (scRNA-seq): In scRNA-seq analysis, clustering trees help identify distinct cell types within tissue samples.
- ✓ Iris Dataset: Clustering trees can be applied to classic datasets like the iris dataset.

Example:

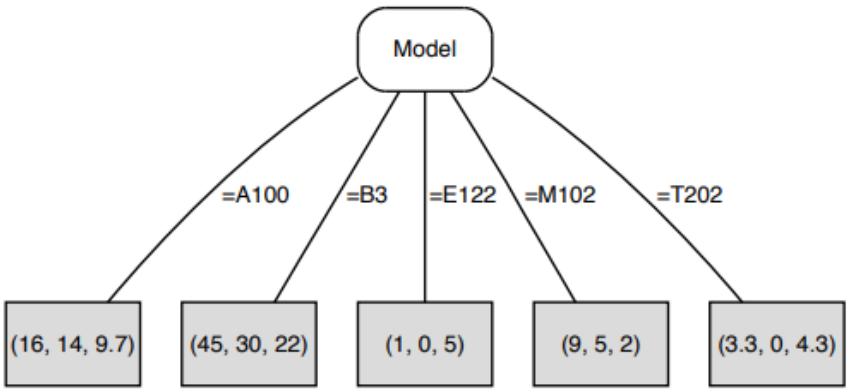
Consider the following dataset:

Model	Condition	Leslie	Price	Reserve	Bids
B3	excellent	no	45	30	22
T202	fair	yes	6	0	9
A100	good	no	11	8	13
T202	good	no	3	0	1
M102	good	yes	9	5	2
A100	excellent	no	18	15	15
T202	fair	no	1	0	3
A100	good	yes	19	19	1
E112	fair	no	1	0	5

$$Mean\ (\mu) = \frac{\sum x_i}{N}$$

$$Variance\ (\sigma^2) = \frac{\sum (x_i - \mu)^2}{N}$$

- ✓ The means of the three numerical features are (12.6, 8.6, 7.9) and their variances are (158, 101.8, 48.8). The average squared Euclidean distance to the mean is then the sum of these variances, which is 308.6.
- ✓ For the A100 cluster these vectors are (16, 14, 9.7) and (12.7, 20.7, 38.2), with average squared distance to the mean 71.6; for the T202 cluster they are (3.3, 0, 4.3) and (4.2, 0, 11.6), with average squared distance 15.8.
- ✓ Using this split we can construct a clustering tree whose leaves are labelled with the mean vectors as follows:



- ✓ A clustering tree learned from the dataset using Euclidean distance on the numerical features.